Plagiarism Detector Report Page

plagiarismdetector.net

Plagiarism Score	Unique Score	AI Detection Score
4% 4% Exact Match 0% Partial Match [Fully Original]	96% High Uniqueness Index	? AI detection not performed. Scan with Plagiarism Detector to view results.

Total Words	986	Total Characters	7540	Plagiarized Sentences	4	Unique Sentences	96%

Plagiarism Sources

No critical matches found. Matched phrases correspond to standard IEEE taxonomy and software engineering terminologies.

References

[1] Z. Wei, J. Sun, et al., 'Advanced Smart Contract Vulnerability Detection via LLM-Powered Multi-Agent Systems,' IEEE TSE, Oct. 2025.

Document Content

3.3 Study of Proposed System The proposed system, LLM-SmartAudit, addresses the critical challenges of smart contract vulnerability detection through an LLM-powered multi-agent conversational framework. Rather than relying on rigid static rule-checkers or error-prone single-agent large language models, the framework decomposes the auditing workflow into specialized collaborative roles: Project Manager (PM), Smart Contract Auditor (SCA), Solidity Programming Expert (SPE), and Smart Contract Counselor (SCC). Department of Computer Science & Design, FISAT. The system operates under two complementary analytical strategies: Broad Analysis (BA) mode and Targeted Analysis (TA) mode. BA mode uses Thought-Reasoning derived from ReAct to explore undocumented security weaknesses across complete contract logic, while TA mode utilizes Buffer-Reasoning derived from Buffer of Thoughts (BoT) to retrieve domain-specific thought templates for 40 pre-established vulnerability scenarios. Through up to three rounds of structured conversational debate and role-swapping to eliminate confirmation bias, the agents achieve consensus. When evaluated on benchmark datasets comprising 110 common vulnerability contracts, 6,454 real-world Code4rena contracts, and 13 critical CVEs, the proposed system demonstrates 98% accuracy on common vulnerabilities and resolves 12 out of 13 CVEs at an average operational cost of ~$0.98 USD per contract. The overall approach connects AST parsing, multi-agent conversational reasoning, and automated report synthesis in a unified pipeline.

3.4 Requirement Specification The following requirements describe the essential computational environment, dependencies, and software resources necessary to deploy and operate the LLM-SmartAudit multi-agent framework. They should not be interpreted as exact hardware specifications but reflect the computational baseline required for parallel token processing and abstract syntax tree (AST) analysis. **3.4.1 Recommended System Requirements** The following requirements describe the general resources needed to support an automated auditing framework: • Processor: A modern multi-core processor capable of handling concurrent network requests and local AST graph generation (Intel Core i5-13400 or higher). • Memory: Sufficient RAM to store and process large smart contract AST representations and message buffer logs (Minimum 16GB, 32GB recommended). • Storage: Adequate storage for contract repositories, Code4rena benchmark sets, vulnerability template libraries, and audit transcripts (Minimum 50GB NVMe SSD). • Operating System: Ubuntu 22.04 LTS (x86_64) or compatible Linux distribution. • Network Connectivity: Stable broadband connection required by the application for accessing frontier LLM APIs (OpenAI, Anthropic, DeepSeek) and remote Ethereum node verification services.

3.4.2 Software Requirements The framework requires software components capable of parsing contract logic, representing abstract syntax trees, managing agent conversation dialogues, and generating structured reports: • Data Ingestion & Parsing: Software for storing, organising, and preprocessing Solidity source code into normalized AST and control-flow representations. Department of Computer Science & Design, FISAT. • Multi-Agent Conversational Framework: Python 3.12 runtime with LangChain / AutoGen agent orchestration modules. • LLM Engine: GPT-4o, DeepSeek-r1, and Claude 3.5 Sonnet accessible via authenticated REST APIs. • Static Baselines: Integration modules for Slither and Mythril to provide comparative baseline verification. • Output Generation: Automated report rendering engine capable of formatting line-level vulnerability traces and concrete patch recommendations into Markdown and PDF formats.

3.5 Functional Requirements The accessibility-aware auditing framework should satisfy the following functional requirements: 1. The system shall parse and ingest raw Solidity smart contract source files, generating structured abstract syntax trees and segmented logic blocks. 2. The system shall deploy specialized agent personas (PM, SCA, SPE, SCC) utilizing Inception Prompting to enforce distinct analytical roles. 3. The system shall support Broad Analysis (BA) mode to perform generalized exploratory scans across arbitrary smart contract logic. 4. The system shall support Targeted Analysis (TA) mode utilizing Buffer of Thoughts (BoT) to evaluate contracts against 40 predefined critical vulnerability templates. 5. The system shall execute structured multi-turn collaborative debates (capped at 3 conversational turns) between SCA and SPE agents to resolve divergent vulnerability scores. 6. The system shall implement bidirectional role-swapping mechanisms during debate to mitigate single-agent cognitive bias and hallucination traps. 7. The system shall synthesize validated agent findings through the Smart Contract Counselor to generate a unified final consensus audit report. 8. The system shall detect machine-auditable flaws including Reentrancy, Integer Overflow/Underflow, Unchecked Send, and Unsafe Delegatecall with high precision. 9. The system shall detect complex contextual logic flaws including Timestamp Dependency, Access Control flaws, and Business Logic errors. 10. The system shall trace exploit paths directly back to specific code lines in Solidity and output concrete, human-readable mitigation steps.

recommendations through the final audit reporting module. **3.6 Non-Functional Requirements** The smart contract auditing framework should satisfy the following non-functional requirements: • Usability: Audit reports must present clear, human-readable vulnerability descriptions, line-level code references, and actionable remediation guidelines so that developers can quickly understand and remediate potential security risks. • Accuracy & Precision: The multi-agent consensus module must achieve higher precision and lower false-positive rates than conventional static analysis tools, maintaining a verified target accuracy of 98% across standard machine-auditable vulnerabilities. • Performance & Latency: The conversational loop must calculate and conclude audits within a practical operational timeframe by bounding agent interaction to a maximum of 3 turns, avoiding infinite dialogue loops or state space explosion. • Cost Efficiency: Operational token expenditures must remain economical, averaging ~$0.21 USD per contract for BA mode and ~$0.98 USD for TA mode. • Robustness & Reproducibility: Repeated executions across identical contracts must exhibit consistent detection stability across independent runs (achieving consistent detection accuracy of 19.4% in BA mode and 54.7% in TA mode across five consecutive trials). • Adaptability: The framework should accommodate different Solidity compiler versions (0.4.x through 0.8.x) and diverse decentralized finance (DeFi) protocol architectures without requiring manual rule reprogramming. • Maintainability: The Buffer of Thoughts (BoT) template database and agent system prompts should be organized modularly so that emerging vulnerability definitions (CVEs) can be incorporated rapidly into future audit runs. • Safety and Inclusiveness: The system should provide comprehensive security verification for novice and experienced Solidity developers alike, ensuring decentralized applications are safeguarded against exploitation without assuming access to expensive commercial auditing firms.

3.7 Summary This chapter analyzed the proof of concept, existing system limitations, and requirements of the LLM-SmartAudit multi-agent framework for smart contract security auditing. The reviewed framework addresses the limitations of conventional static analysis by combining specialized agent personas, dual-mode analysis strategies, and conversational consensus mechanisms. The proposed approach connects AST parsing with multi-agent debate and automated report synthesis, ensuring higher precision, minimal hallucinations, and affordable verification. Its effectiveness depends on the quality and completeness of prompt templates and LLM reasoning depth. Overall, the framework provides an interpretable and robust approach to improving smart contract reliability by incorporating multi-agent collaborative intelligence into automated software vulnerability auditing. Department of Computer Science and Design, FISAT.